
第 4 章 故障解析

本章节分享内容都源于用户的现场故障。为您分析故障原因和解决定位故障的过程，将问题的相关知识点加以融合说明。为您呈现一线 DBA 最真实的工作写照。希望在看过作者们的故障分析后，有助于读者快速定位解决现场故障。

4.1 MySQL 5.7 使用 mysqldump 重要闭坑事项

作者：王向

背景

笔者在一次处理客户 MySQL 问题时遇到客户的 MySQL 的 sys 库不能用了并抛出一下错误：

```
mysql> SELECT * FROM sys.processlist;
ERROR 1356 (HY000): View 'sys.processlist' references invalid table(s) or column(s) or function(s) or definer/invoke of view lack rights to use them
```

首先，这个问题其实并不难解决，但是这个问题引发的现象倒是挺有意思。

排查常见问题

先定位几个常见问题：

1. 权限不够；
2. sys 库 functions 和 procedures 丢失；
3. mysqldump 全备后跨版本恢复【会发生问题 2 的现象】；
4. mysql 升级没有执行 mysql_upgrade【会发生问题 2 的现象】；

首先排查权限问题是否有权限。

```
mysql> SHOW GRANTS FOR root@'localhost';
+-----+
| Grants for root@localhost |
+-----+
| GRANT ALL PRIVILEGES ON *.* TO 'root'@'localhost' WITH GRANT OPTION |
| GRANT PROXY ON ''@' TO 'root'@'localhost' WITH GRANT OPTION |
+-----+
2 rows in set (0.00 sec)
```

明显并不是，接着排查是否是 sys 库相关的 functions 和 procedures 丢失了？

```
mysql> SELECT * FROM mysql.proc;
Empty set (0.00 sec)

mysql> SHOW PROCEDURE STATUS WHERE Db = 'sys';
Empty set (0.00 sec)

mysql> SHOW FUNCTION STATUS WHERE Db = 'sys';
Empty set (0.00 sec)
```

sys 库 functions 和 procedures 丢失了，那不就是问题 3 就是问题 4，甩给 mysqldump 全备和升级没有执行 mysql_upgrade。

带着疑问于开始漫长的排查过程。经过对客户的刨根问题，发现并没有上述情况的发生。用户备份习惯都是全备（-A），且都是备份恢复后出现 sys 库 **ERROR 1356**，检查用户 MySQL 环境主要几大版本分布 MySQL 5.7.13, 5.7.25, 5.7.28。于是把问题定位到了 mysqldump 的备份上。

先备份还原一把看看

笔者强烈认为是客户跨版本造成的，给客户来点证据。先验证一波同版本 MySQL 使用 mysqldump 全备恢复后，到底会不会出现 sys 库 ERROR 1356。

备份前先检测一波 sys 库，确认完全 OK 后开整。

```
mysql> SELECT * FROM sys.version;
+-----+-----+
| sys_version | mysql_version |
+-----+-----+
| 1.5.2       | 5.7.31-log    |
+-----+-----+
1 row in set (0.00 sec)

mysql> SELECT * FROM sys.processlist;
ERROR 1356 (HY000): View 'sys.processlist' references invalid table(s) or column(s) or function(s) or definer/invoke of view lack rights to use them

mysql> SELECT COUNT(*) FROM mysql.proc;
+-----+
| COUNT(*) |
+-----+
|        48 |
+-----+
1 row in set (0.00 sec)
```

使用我们经常用的那坨命令备份所有库。

```
mysqldump --all-databases --set-gtid-purged=OFF \  
--master-data=2 --single-transaction --routines \  
--events --triggers --max_allowed_packet=256M > all.sql
```

备份完毕后我们开始恢复数据。

```
mysql -uroot -S /tmp/mysql.sock < all.sql
```

恢复完毕后，检测一波 sys 库。

```
mysql> SELECT * FROM sys.processlist;  
ERROR 1356 (HY000): View 'sys.processlist' references invalid table(s) or column(s) or  
function(s) or definer/invoker of view lack rights to use them  
  
mysql> SELECT COUNT(*) FROM mysql.proc;  
+-----+  
| COUNT(*) |  
+-----+  
|          0 |  
+-----+  
1 row in set (0.00 sec)  
  
mysql> SHOW PROCEDURE STATUS WHERE Db = 'sys';  
Empty set (0.00 sec)  
  
mysql> SHOW FUNCTION STATUS WHERE Db = 'sys';  
Empty set (0.00 sec)
```

啪啪啪打脸，竟然同版本也会出现？那究竟是什么问题？

再看看其它版本

经过对 MySQL 5.7.13, 5.7.21, 5.7.25, 5.7.28, 5.7.31 等几个版本测试全备躺枪。奇怪的现象是他们唯一的共性就是无论怎么备份怎么还原只要使用了 **--all-databases (-A)** 就报 **ERROR 1356**。这不禁让笔者陷入了沉思。

寻找突破点

既然通用规律只有使用 **--all-databases (-A)** 会 **ERROR 1356**，那就看看他到底备份了什么东西。于是喊上同事一起 less 看了下，上下扫了两眼。突然发现：

1. 备份 SQL 文件里 DROP 掉了 **mysql.proc**;
2. 后 CREATE 了一个新的 **mysql.proc**;
3. **LOCK TABLES** 和 **UNLOCK TABLES** 中间居然没有备份 **CREATE ROUTINE** 任何数据？

这不就是相当于每次导入全备都给我一个没有任何 sys schema routines 的全新 **mysql.proc** 表？那这不就异常的尴尬？

```
--
-- Table structure for table `proc`
--

DROP TABLE IF EXISTS `proc`;
/*!40101 SET @saved_cs_client      = @@character_set_client */;
/*!40101 SET character_set_client = utf8 */;
CREATE TABLE `proc` (
  `db` char(64) CHARACTER SET utf8 COLLATE utf8_bin NOT NULL DEFAULT '',
  `name` char(64) NOT NULL DEFAULT '',
  `type` enum('FUNCTION','PROCEDURE') NOT NULL,
  `specific_name` char(64) NOT NULL DEFAULT '',
  `language` enum('SQL') NOT NULL DEFAULT 'SQL',
  `sql_data_access` enum('CONTAINS_SQL','NO_SQL','READS_SQL_DATA','MODIFIES_SQL_DATA')
NOT NULL DEFAULT 'CONTAINS_SQL',
  `is_deterministic` enum('YES','NO') NOT NULL DEFAULT 'NO',
  `security_type` enum('INVOKER','DEFINER') NOT NULL DEFAULT 'DEFINER',
  `param_list` blob NOT NULL,
  `returns` longblob NOT NULL,
  `body` longblob NOT NULL,
  `definer` char(93) CHARACTER SET utf8 COLLATE utf8_bin NOT NULL DEFAULT '',
  `created` timestamp NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  `modified` timestamp NOT NULL DEFAULT '0000-00-00 00:00:00',
  `sql_mode`
set('REAL_AS_FLOAT','PIPES_AS_CONCAT','ANSI_QUOTES','IGNORE_SPACE','NOT_USED','ONLY_FUL
L_GROUP_BY','NO_UNSIGNED_SUBTRACTION','NO_DIR_IN_CREATE','POSTGRESQL','ORACLE','MSSQL',
'DB2','MAXDB','NO_KEY_OPTIONS','NO_TABLE_OPTIONS','NO_FIELD_OPTIONS','MYSQL323','MYSQL4
0','ANSI','NO_AUTO_VALUE_ON_ZERO','NO_BACKSLASH_ESCAPES','STRICT_TRANS_TABLES','STRICT_
ALL_TABLES','NO_ZERO_IN_DATE','NO_ZERO_DATE','INVALID_DATES','ERROR_FOR_DIVISION_BY_ZER
O','TRADITIONAL','NO_AUTO_CREATE_USER','HIGH_NOT_PRECEDENCE','NO_ENGINE_SUBSTITUTION','
PAD_CHAR_TO_FULL_LENGTH') NOT NULL DEFAULT '',
  `comment` text CHARACTER SET utf8 COLLATE utf8_bin NOT NULL,
  `character_set_client` char(32) CHARACTER SET utf8 COLLATE utf8_bin DEFAULT NULL,
  `collation_connection` char(32) CHARACTER SET utf8 COLLATE utf8_bin DEFAULT NULL,
  `db_collation` char(32) CHARACTER SET utf8 COLLATE utf8_bin DEFAULT NULL,
  `body_utf8` longblob,
  PRIMARY KEY (`db`,`name`,`type`)
) ENGINE=MyISAM DEFAULT CHARSET=utf8 COMMENT='Stored Procedures';
/*!40101 SET character_set_client = @saved_cs_client */;
--
```

```
-- Dumping data for table `proc`  
--  
  
LOCK TABLES `proc` WRITE;  
/*!40000 ALTER TABLE `proc` DISABLE KEYS */;  
/*!40000 ALTER TABLE `proc` ENABLE KEYS */;  
UNLOCK TABLES;
```

真相大白

在官方文档【sys-schema-usage】

<https://dev.mysql.com/doc/refman/5.7/en/sys-schema-usage.html>

页面有这样一段话（这里直接引用官方原文）：

However, those statements display the definitions in relatively unformatted form. To view object definitions with more readable formatting, access the individual **.sql** files found under the **scripts/sys_schema** in MySQL source distributions. Prior to MySQL 5.7.28, the sources are maintained in a separate distribution available from the **sys** schema development website at <https://github.com/mysql/mysql-sys>.

Neither **mysqldump** nor **mysqlpump** dump the sys schema by default. To generate a dump file, name the sys schema explicitly on the command line using either of these commands:

sys:

<https://dev.mysql.com/doc/refman/5.7/en/sys-schema.html>

mysqldump:

<https://dev.mysql.com/doc/refman/5.7/en/mysqldump.html>

```
mysqldump --databases --routines sys > sys_dump.sql  
mysqlpump sys > sys_dump.sql
```

To reinstall the schema from the dump file, use this command:

```
mysql < sys_dump.sql
```

官方文档明确的告诉我们不会备份 sys 库。但在使用 **mysqldump** 在执行 **--all-databases** 会清空 mysql.proc 导致 sys 无法正常使用；这是一个 BUG，并且只存在于 MySQL 5.7.x ！

BUG 连接：

-
- <https://bugs.mysql.com/bug.php?id=86807>
 - <https://bugs.mysql.com/bug.php?id=92631>
 - <https://bugs.mysql.com/bug.php?id=83259>
 - <https://github.com/mysql/mysql-server/commit/ded3155def2ba3356017c958c49ff58c2cae1830>

解决方案和使用场景

针对这个 BUG 整理了 4 个解决方案可供参考，根据实际环境场景进行选择使用。

1、mysql_upgrade install or upgrade sys schema

这个方案适用于 sys 库已经因为 mysqldump 导入而损坏的情况下使用。

```
# 删除 sys schema (An error occurs if a sys schema exists but has no version view)
mysql> DROP DATABASE sys;

# 这个时候 sys schema 不应该存在
mysql> SHOW DATABASES;

# 最后，执行 mysql_upgrade sys schema 以恢复正常
mysql_upgrade --upgrade-system-tables --skip-verbose --force

mysql> SHOW DATABASES;
mysql> SELECT COUNT(*) FROM mysql.proc;
```

注意：mysql_upgrade 在修理 sys 库的同时，还修理 mysql 库和用户库表（期间加锁且速度一般），有极小可能会误伤；使用 mysql_upgrade 的时候要加上 --upgrade-system-tables，不然会扫描用户库表。

2、全备时同时备份 sys 库

这个方案适用于需要还原的数据库，sys 库也不太正常的情况下使用；在全备后额外再备份一份 sys 库用于修复。

```
mysqldump -A --set-gtid-purged=OFF --master-data=2 --single-transaction --routines --events --triggers > all.sql
mysqldump --databases --routines sys > sys_dump_`mysql -V|awk '{print $5}'|cut -b 1-6`.sql
```

注意：不适用于做主从时使用它。

3、使用 databases 全备

这个方案适用于所有场景的全备需求，100% 安全。

```
select_databases="
```

```
SELECT
    GROUP_CONCAT(schema_name SEPARATOR ' ')
FROM
    information_schema.schemata
WHERE
    schema_name NOT IN ('performance_schema','information_schema');
```

```
databases=`mysql -NBe "$select_databases"`
mysqldump --set-gtid-purged=OFF --master-data=2 \
--single-transaction --routines --events --triggers \
--max_allowed_packet=256M --databases > all.sql
```

4、使用 `mysql-sys` 开源代码

如果你的数据库 `sys` 全部中招了，又是生产库。那你只能用这个方法：

`mysql-sys`:

<https://github.com/mysql/mysql-sys>

中记录了 `sys` 库的创建语句将文件下载到本地，然后根据数据库版本，执行以下命令即可。

安装前操作，内容是禁用掉 `sql_log_bin`，不记录到日志中。

```
mysql> source before_setup.sql
```

创建 `sys` 库，实际会调用其他文件夹中的 `sql` 语句

来进行表、视图、存储过程、触发器的创建

```
mysql> source sys_57.sql
```

安装后的操作，内容是将 `sql_log_bin` 恢复到操作前的状态

```
mysql> source after_setup.sql
```

【加餐 1】试试 MySQL 8

测试 MySQL 8.0.0 至 MySQL 8.0.20 全系列不受影响，具体原因是从 MySQL 8.0.0 起就移除了 `mysql.proc` 这张表。具体查阅官方文档：

1. data-dictionary-usage-differences:

<https://dev.mysql.com/doc/refman/8.0/en/data-dictionary-usage-differences.html>

2. news-8-0-0:

<https://dev.mysql.com/doc/relnotes/mysql/8.0/en/news-8-0-0.html>

Previously, tables in the `mysql` system database were visible to DML and DDL statements. As of MySQL 8.0, data dictionary tables are invisible and cannot be modified or queried directly. However, in most

cases there are corresponding INFORMATION_SCHEMA tables that can be queried instead. This enables the underlying data dictionary tables to be changed as server development proceeds, while maintaining a stable INFORMATION_SCHEMA interface for application use.

【加餐 2】如果还有疑问？

那就顺便看一眼 mysqldump 的源码吧（这个源码的设计也挺有意思，准备放到后面的文章里面），先过一眼这个变量。

```
/**
 * First mysql version supporting the information schema.
 */
#define FIRST_INFORMATION_SCHEMA_VERSION 50003
/**
 * Name of the information schema database.
 */
#define INFORMATION_SCHEMA_DB_NAME "information_schema"
/**
 * First mysql version supporting the performance schema.
 */
#define FIRST_PERFORMANCE_SCHEMA_VERSION 50503
/**
 * Name of the performance schema database.
 */
#define PERFORMANCE_SCHEMA_DB_NAME "performance_schema"

/**
 * First mysql version supporting the sys schema.
 */
#define FIRST_SYS_SCHEMA_VERSION 50707 /* 最早出现 sys schema 的 MySQL 版本 5.7.7 */

/**
 * Name of the sys schema database.
 */
#define SYS_SCHEMA_DB_NAME "sys"
```

dump 所有库表（--all-databases）的源码：

```
.....
/* 执行 dump_all_databases 的条件 */
if (opt_alldbs)
{
    if (!opt_alltspcs && !opt_notspcs)
        dump_all_tablespace();
}
```

```
    dump_all_databases();
}
.....

/* dump_all_databases */
static int dump_all_databases()
{
    MYSQL_ROW row;
    MYSQL_RES *tableres;
    int result=0

    /* 获取所有数据库: SHOW DATABASES */
    if (mysql_query_with_error_report(mysql, &tableres, "SHOW DATABASES"))
        return 1;
    while ((row= mysql_fetch_row(tableres)))
    {

        /* 排除 information_schema */
        if (mysql_get_server_version(mysql) >= FIRST_INFORMATION_SCHEMA_VERSION &&
            !my_strcasecmp(&my_charset_latin1, row[0], INFORMATION_SCHEMA_DB_NAME))
            continue;

        /* 排除 performance_schema */
        if (mysql_get_server_version(mysql) >= FIRST_PERFORMANCE_SCHEMA_VERSION &&
            !my_strcasecmp(&my_charset_latin1, row[0], PERFORMANCE_SCHEMA_DB_NAME))
            continue;

        /* 排除 sys */
        /* 检查当前 MySQL 的版本是否 >= 最早支持 SYS_SCHEMA 的版本号。&& row[0] 为
SYS_SCHEMA_DB_NAME 就跳过, 不进行备份*/
        if (mysql_get_server_version(mysql) >= FIRST_SYS_SCHEMA_VERSION &&
            !my_strcasecmp(&my_charset_latin1, row[0], SYS_SCHEMA_DB_NAME))
            continue;

        if (is_ndbinfo(mysql, row[0]))
            continue;

        /* dump 库中所有表 */
        /* 逐一 dump 每个表 dump_all_tables_in_db */
        if (dump_all_tables_in_db(row[0]))
            result=1;
    }
    .....
}
```

备份 functions 和 procedures 的源码:

```
/** 此处 --all-databases sys 库不会传入 dump_routines_for_db 这个函数。
    所以函数里面的备份过程跳过了 sys 库，也就造成了.sql 文件里 mysql.proc 没有 CREATE ROUTINE sys
    库的现象 */
static uint dump_routines_for_db(char *db)
{
    .....
    /* 0, retrieve and dump functions, 1, procedures */
    for (i= 0; i <= 1; i++)
    {
        /* 执行 SHOW FUNCTION/PROCEDURE STATUS WHERE Db = xx, 获取所有 functions 和 procedures
        */
        my_snprintf(query_buff, sizeof(query_buff),
                    "SHOW %s STATUS WHERE Db = '%s'",
                    routine_type[i], db_name_buff);

        if (mysql_query_with_error_report(mysql, &routine_list_res, query_buff))
            DBUG_RETURN(1);

        if (mysql_num_rows(routine_list_res))
        {
            while ((routine_list_row= mysql_fetch_row(routine_list_res)))
            {
                routine_name= quote_name(routine_list_row[1], name_buff, 0);
                DBUG_PRINT("info", ("retrieving CREATE %s for %s", routine_type[i],
                                    name_buff));

                /* 执行 SHOW CREATE FUNCTION/PROCEDURE xxx, 获取所有 functions、procedures 创建语句
                */
                my_snprintf(query_buff, sizeof(query_buff), "SHOW CREATE %s %s",
                            routine_type[i], routine_name);

                .....
            }
        }
    }
}
```